

バイブコーディングから 仕様駆動開発へ

神経衰弱を「いい感じに」作る。

ポーカーを、要件を書いてから作る。



前回の復習

サイクルは今日も変わりません

01

Ask

@ファイル 質問

内容を理解する

02

Agent

@ファイル 依頼
+ 完了条件

変えてもらう

03

diff

変更を読む

何が変わったか

04

Keep / Undo

採用 or 取り消し

自分で決める

変わるのは、何を渡すかです。サイクルそのものは今日も同じ。

今日やること

2つのゲームを作ります

1つ目は簡単なゲームを何も決めずに。2つ目は決めることが桁違いに多いゲームを、要件を書いてから。

1つ目

神経衰弱

メモリーマッチ

何も決めずに
「いい感じに作って」

バイブコーディング ・ 第2章

撮影待ち

できた神経衰弱
(見本)

2つ目

5カードドローパーカー

Five-Card Draw

先に要件を
書いてから作る

仕様駆動開発 ・ 第5章・第6章

撮影待ち

できたポーカー
(見本)

どちらのやり方が良いかではなく、どちらをいつ使うかを持ち帰ってもらいます。

今日の約束

3つだけ先に言っておきます

01

完成させなくていい

2つ目は途中で終わっても構いません。終わった人は挑戦卒へ

02

1つ目の粗さは気にしなくていい

わざと雑に頼みます

03

詰まったら手を挙げる

隣の人がいれば、隣の画面を見るのも早いです

90分の流れ

タイムテーブル

時刻	章	内容	主体
0:00	第1章 今日のゴール	2つのゲームを作ると宣言する (5分)	講師
0:05	第2章 バイブコーディングで作る	神経衰弱を例の3行で作る (12分)	全員
0:17	第3章 見比べる	見本と自分のものを見比べる (5分)	全員
0:22	第4章 後出し要求	追加要求を神経衰弱に入れる (10分)	全員
0:32	(まとめ) ここまでで分かったこと	バイブコーディングの回収 → 仕様駆動へ (3分)	講師
0:35	第5章 要件を書く	ポーカーの要件を要件作成スキルで書く (15分)	全員
0:50	第6章 タスクに割って、まず2つ作る	タスク分解 → 手で2つ実装 (15分)	全員
1:05	第7章 開発ルールを決めて、効かせる	ルールを書く → 2行を書かずに続きを作る (15分)	全員
1:20	第8章 持ち帰るもの	使い分けの判断・次回予告 (10分)	講師

手を動かすのは **72分 / 90分**。第3章・第4章は**すでに 5分 / 10分まで絞ってあるので**、押してもこれ以上削りません。

CURSOR 実践コース 第 2 回 / 全 4 回

CHAPTER 01 ・ 0:00 5分

今日のゴール

2つのゲームを作ります。1つ目は簡単なゲームをバンプコーディングで、2つ目は難しいゲームを要件から。

今日のゴール

4段のはしご

01 バイブコーディングで動くものを出す

第2章

02 同じ依頼から全員が別のものを作ったことに気づく

第3章

03 決めることの多いゲームを、要件を書いてから作らせる

第5章・第6章

04 後から来た要求を、要件に足して通す

挑戦

第6章

01～03 まで届けば達成です。04 は挑戦枠なので、届かなくても失敗ではありません。

用語

この2語で通します

言葉	指すもの
バИБコーディング	何も決めずに頼むやり方の名前。今日の1つ目（神経衰弱）はこれで作ります
仕様駆動開発	先に要件を書いてから頼むやり方の名前。今日の2つ目（ポーカー）はこれで作ります

どちらも「やり方」の名前です。できあがったものは、ゲームの名前でそのまま呼びます（「神経衰弱」「ポーカー」）。略して「バИБ」とは言いません。

第1章 コラム

今日は何度も Agent を回します

先に30秒だけ、使用量の話をしておきます。

Auto でも	振り分けられた そのモデルの価格 で課金されます
どのプランにも	含まれている使用量があります
使い切ったら	従量（後払い）で使い続けるか、プランを上げるかです

Auto の方針（Cost / Balance / Intelligence）は、**どのモデルへ振り分けるかを変えるもの**であって、割引ではありません。金額はこの資料に書きません。判断が要るときは公式の料金ページを見てください。

CURSOR 実践コース 第 2 回 / 全 4 回

CHAPTER 02 ・ 0:05 12分

バイブコーディングで作る

何も決めずに頼むと、どこまで出るか。全員が自分の神経衰弱を持ちます。

第2章 受講者がやること

神経衰弱を作らせる（12分）

- 1 `session02/` という新しいフォルダを作る（1分）
- 2 新しいチャットを開いて、下の3行を送る（10分）
- 3 出てきたらブラウザで開く（1分）
サイドバーで右クリック → **Open In Browser**

神経衰弱ゲームを作って。
HTML + JS で、ブラウザで遊べるようにして。
いい感じに。

「いい感じに」は、第1回で避ける言い方として挙げたフレーズです。わざと雑に頼んでいます。モデルは **Auto** のままで構いません。

撮影待ち

神経衰弱の依頼を
入れた状態（送信前）

第2章 見本

数分で、動くものが出ます

見本 ・ 同じものにはなりません

撮影待ち

できた神経衰弱
(受講者が作ったもの)

速い。動くものがすぐ出る。ここまではバンプコーディングの得意分野です。

第2章 詰まったら

動かないときは、Agent に見てもらおう

Cursor のブラウザは **Agent** が操作できます。開く・クリックする・コンソールのエラーを読むまで頼めます。

いま作ったゲームをブラウザで開いて、
コンソールにエラーが出ていたら原因を教えて。

OS のブラウザで直接開くと動かないことがあります。Cursor の内蔵ブラウザで開いてください。

ブラウザ操作には承認が要ります（既定は操作のたびに確認する「手動承認」）。授業ではそのまま構いません。

CURSOR 実践コース 第 2 回 / 全 4 回

CHAPTER 03 ・ 0:17 5分

見比べる

同じ3行から、違うものが出たことを確かめます。比べる相手はスライドの見本なので、隣がいなくても成立します。

第3章 ★

同じ3行から、違うものが出ます

1回目

撮影待ち

できた神経衰弱（1回目）

2回目

撮影待ち

同じ3行で作った別のもの（2回目）

わたしが、同じモデルに同じ3行を2回送っただけです。それでも別のものが出ます。あなたのものは、このどちらとも違うはずです。

第3章 受講者がやること

見本2枚と、自分のものを比べる

前のページの見本2枚と、自分のものを比べてください。講師は見本を出したままにします。

比べるもの	見本 1回目	見本 2回目	自分
カードは何枚か			
表の絵柄は何か（数字 / 絵文字 / 色）			
スコアや手数は表示されるか			
難易度の選択はあるか			
見た目（色・レイアウト）			

隣の人がいれば、同じ表を隣のものでも埋めてください。いなければ見本だけで構いません。

第3章 受講者がやること

頼んでいないものを数える（1分）

自分が送った依頼は**3行だけ**でした。それ以外に付いてきたものを挙げてください。

頼んだもの

神経衰弱であること
HTML + JS で動くこと
いい感じであること

頼んでいないのに付いてきたもの

カードの枚数
絵柄
スコア・手数
難易度の選択
色・レイアウト

スコアも難易度も、誰も頼んでいません。指示したものと、AI が決めたものの区別がつかない。

CURSOR 実践コース 第 2 回 / 全 4 回

CHAPTER 04 ・ 0:22 10分

後出し要求

この回の山場です。発注者から追加要求が来たとき、いま作った神経衰弱で何が起きるか。

第4章 受講者がやること

新しいチャットを開く (1分)

1 いまのチャットではなく、新しいチャットを開く

いまのチャットには「さっき何を作ったか」の文脈が残っています

2 新しいチャットは、その文脈を持っていません

他人に引き継ぐのと同じ状況になります

ここを飛ばすと、この章が成立しません。「文脈を持たない相手に渡す」という状況を作るための操作です。

撮影待ち

新しいチャットを開く場所

第4章 発注者から

追加の要望が2つ来ました

- ① ペアが成立したら、もう一度めくれるようにしてください。
- ② 3回目以降のミスからは、裏に戻るまでの待ち時間を 0.5 秒に短くしてください。

この2つは、AI が神経衰弱の常識からは推測しないルールです。だから、いまあなたが作ったものには1つも入っていません。

第4章 受講者がやること

追加要求を実装させる（6分）

前のページの2つを、そのまま送ってください。

```
@session02/
```

- ① ペアが成立したら、もう一度めくれるようにして。
- ② 3回目以降のミスからは、裏に戻るまでの待ち時間を 0.5 秒にして。
他の挙動は変えないで。

新しいチャットで送ります。いまのチャットの続きではありません。

第4章 受講者がやること

記録する（3分）

記録すること	答え
変更行数（diff の +/-）	
既存の機能が壊れたか	
正しく実装されたか、自分で判定できたか	

3行目が今日いちばん大事な質問です。

第4章

「3回目以降が 0.5 秒になっているか」を、
どうやって確かめましたか。

動かしても分かりません。0.5 秒と 1 秒は、目で見分けられないからです。
確かめるには、コードを開いてタイマーの数字を探すしかありません。

第4章

さっき、新しいチャットを開きましたね。

あのチャットの AI は、あなたが何を作ろうとしたのか**知りません**。

コードを読んで、**推測して**直しました。

カードが8枚なのは、あなたが決めたのか AI が決めたのか

戻るまでが1秒だったのは、意図なのか、たまたまなのか

どこにも書いていないので、読んだ側には分かりません。

いま渡した相手が AI だけです。明日のあなたも、隣の席の人も、同じ立場になります。

第2～4章 まとめ

渡したものと、返ってきたもの

第2章で何が起きたのかを、数えて確認します。

	中身
あなたが渡したもの	3行（ゲームの名前 / HTML + JS / 「いい感じに」）
あなたが決めたこと	神経衰弱であること。ブラウザで動くこと。以上
返ってきたもの	遊べるゲーム 1本（数分）
AI が決めたこと	カードの枚数・絵柄・並べ方・色・スコアの有無・難易度・戻るまでの秒数……

あなたが決めたことより、AI が決めたことの方がずっと多い。

第2～4章 まとめ

一言でいうと

**バンプコーディングは、速い。
ただし コントロールできない。**

第2章の「速い」は強みです。残りの3つは、全部これ1つに畳めます。

毎回違うものが出た

→ 狙えない

AI が決めたものと区別がつかない

→ 把握できない

入ったかどうか判定できない

→ 確かめられない

狙えて、把握できて、確かめられる。この3つが揃って初めてコントロールです。

第2～4章 まとめ

なぜコントロールできないのか

決めていないからです。

決めていなかったこと	だから
カードの枚数も絵柄も決めていない	狙えない
何を頼んだかの記録がない	把握できない
「どうなったら OK か」を決めていない	確かめられない

第4章で言われたのは「0.5 秒にして」だけでした。「どうなったら OK か」は誰も決めていません。だから動かしても分かりません — 0.5 秒と 1 秒は目で見分けられないからです。画面が答えを見せてくれないんです。

第2～4章 まとめ

だから、先に書く

コントロールを取り戻すために、先に決めます。

	パイプコーディング	要件を書いてから
狙う	決めたことは頭の中にある	決めたことが文字になっている
確かめる	「どうなったら OK か」が無い	完了条件を、画面で確かめられる形で書く
渡す	続きは、作った本人にしか頼めない	他の人にも渡せる

第5章 要件を書く（今度は、決めることが桁違いに多いゲームで）

第6章 要件から作る

第7章 開発ルールを決める

パイプコーディングが悪いわけではありません。神経衰弱は数分で出ました。あれは強みです。変わるのは規模と、決めごとの量です。

CURSOR 実践コース 第 2 回 / 全 4 回

CHAPTER 05 ・ 0:35 15分

要件を書く

2つ目のゲームです。神経衰弱よりずっと難しいものを、要件から作ります。

第5章 解説

Skill は「この種の作業の手順書」

毎回同じ手順を書かなくて済むようになります。置き場所は決まっています。

種類	パス	効く範囲
プロジェクト	<code>.cursor/skills/<名前>/SKILL.md</code>	このリポジトリだけ
個人	<code>~/.cursor/skills/<名前>/SKILL.md</code>	自分の全プロジェクト

呼び方	効く範囲	操作
自動	Agent が「使うべき」と判断したとき	何もしない（description で決まる）
スラッシュ	そのメッセージ1回だけ	入力欄で / → Skill 名を選ぶ
Custom Mode	セッション全体	Alt+Enter（Mac は Option+Enter）

今日使う2つは、リポジトリに入っています（`git pull`）。今日は「スラッシュ」で呼びます。

第5章 解説

中身はただの Markdown

頭に2行の設定が付いているだけです。

```
---
name: requirements
description: 作りたいものを対話で聞き取り、
  4項目の要件にまとめるときに使う
---

# 要件をつくる

(ここから下は、ふつうの手順書)
```

項目	役割
name	小文字・数字・ハイフンのみ。親フォルダ名と一致させる
description	ここがいちばん大事。Agent はこれを見て「この会話で使うか」を判断します

必須はこの2つだけです。

第5章 解説

今日使う2つの Skill

Skill	何をするか	何をしないか
requirements	作りたいものを聞き取り、やること / 画面 / 操作 / やらないことにまとめる	コードを書かない
task-breakdown	要件を、完了条件付きのタスクに 3〜5個に割り、tasks.md に保存する	コードを書かない

どちらも実装しません。「決める」ことだけをやる道具です。

第5章 講師デモ ★

ポーカーを「いい感じに」で頼むと

依頼はこの3行だけ。第2章の神経衰弱とまったく同じ頼み方です。

ポーカーゲームを作って。
HTML + JS で、ブラウザで遊べるようにして。
いい感じに。

見本 ・ 同じものにはなりません

撮影待ち

「いい感じに」で作ったポーカー
(CREDIT / BET / 配当表・対戦相手なし)

第5章 講師デモ

動いています。でも、これは何でしょう

頼んでいないのに付いてきたもの

チップ (CREDIT 1000)

ベット額の選択

9段階の配当表

頼んだつもりなのに無いもの

対戦相手

これはあなたが作りたかったポーカーですか。

対戦相手はどこにいますか。

チップとベットは、誰が頼みましたか。

第5章 解説

要件に必ず入れる4項目

項目	中身
やること	成立するために必要なこと
画面	何が見えて、何が押せるか
操作	ユーザーが何をするとどうなるか
やらないこと	今回は作らないと決めるもの

4つ目が今日の効きどころです。AI は書いていないことを勝手に足してくるので、先に禁止しておきます。

第5章 解説

ポーカーで「決めないと割れる」もの

神経衰弱と違って、ポーカーは**決めることが多い**ゲームです。

決めないと割れるもの	例
対戦相手	そもそも相手はいるのか。いるなら CPU は何人で、どう交換するのか
役の強さの順	10段階のどこまで実装するか
同じ役どうしの比較	ワンペア同士は、どちらが勝つのか
交換	何枚まで / 何回まで / 0枚を選べるか
ベット	チップを賭けるのか、賭けないのか

放っておくと AI が全部決めます。決まったものが自分の意図と合っているかは、要件が無いと確かめようがありません。

第5章 配布

役の強さ（強い順）

枠が付いているカードが、その役をつくっているカードです。役の名前は英語表記に統一します。

- 1 **Royal Flush**
ロイヤルストレートフラッシュ
同じマークの 10・J・Q・K・A



- 2 **Straight Flush**
ストレートフラッシュ
同じマークで数字が5枚連続



- 3 **Four of a Kind**
フォーカード
同じ数字が4枚



- 4 **Full House**
フルハウス
同じ数字3枚 + 2枚



- 5 **Flush**
フラッシュ
5枚とも同じマーク



- 6 **Straight**
ストレート
数字が5枚連続



- 7 **Three of a Kind**
スリーカード
同じ数字が3枚



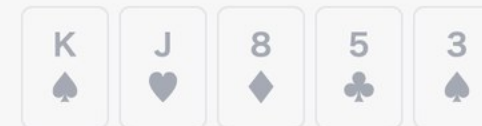
- 8 **Two Pair**
ツーペア
同じ数字2枚が2組



- 9 **One Pair**
ワンペア
同じ数字が2枚



- 10 **High Card**
ハイカード
どれにも当てはまらない



この表のどこまでを作るかは、あなたが要件で決めます。全部作らなくて構いません。

第5章 受講者がやること

要件作成スキルを呼んで、埋める

① 新しいチャットで、入力欄に `/` を打って **requirements** を選びます (2分)。

```
/requirements 5カードドロポーカーを作りたい
```

一覧に出ない人は `git pull` が済んでいません。

② スキルが聞いてくることに答えます (8分)。埋めるのは4つ。

- やること — ゲームとして成立するために必要なこと
- 画面 — 何が見えて、何が押せるか
- 操作 — ユーザーが何をするとどうなるか
- やらないこと — 今回は作らないと決めるもの

撮影待ち

`/` でスキル一覧が出た状態

第5章 効きどころ

「やらないこと」に書けるもの

思いつかない人は、ここから選んでください。

ベット・チップ・掛け金

ブラフ・CPU の戦略

3人以上のプレイヤー

ジョーカー

アニメーション（カードをめくる演出）

成績の保存

AI は書いていないことを勝手に足します。先に禁止しておけば、足されません。

第5章 発注者から

もう1つ追加です

勝敗が出たら、その役をつくっているカードに
枠を付けてください。
役に関係ないカードは灰色にしてください。

要件に1行足すだけで対応してください。コードは触りません。

CURSOR 実践コース 第 2 回 / 全 4 回

CHAPTER 06 ・ 0:50 15分

タスクに割って、まず2つ作る

要件を、1つずつ渡せる大きさに割ります。ここではまず2つだけ。残りは第7章で、ルールを効かせてから続けます。

第6章 解説

1回の依頼に、1つだけ

第1回で「曖昧な依頼は曖昧な結果になる」をやりました。タスク分解は、その具体的なやり方です。

【やりたいこと】一言

【対象】@ファイル or フォルダ

【制約】壊したくないもの

【完了条件】何があれば終わりか

完了条件があると	返ってきたものを判定できます
長くなったら	新しいチャットに切り替える

第4章で「判定する基準が自分の頭の中にしかなかった」のと、ここが対になります。

第6章 解説

Keep の前に見るもの

サイクルは第1回と同じです。増えるのは1つだけ。

	第1回	今日
依頼したあと	diff を読む	diff を読む
Keep の前に	意図した箇所だけ変わったか	+ 完了条件を満たしたか
判定の基準	自分の目	書いてある完了条件

第4章で「合っているかを自分の記憶としか照らせなかった」のは、**照らす先が無かった**からです。今日は書いてあります。

新しく作らせたときは、diff が全部が追加行になります。そこで見るのは「動くか」と「ファイルが増えすぎていないか」だけ。既存に足させたとき（第4章、タスク2つ目以降）が、第1回と同じ読み方です。

第6章 受講者がやること

フォルダを作って、タスクに割る

1 作業フォルダを作る（1分）

`session02-spec/`。神経衰弱とは混ぜません

2 タスクに割る（4分）

`/task-breakdown` に、書いた要件を渡す

3〜5個に割れば十分です。多すぎたら削ってください。10個以上に割れたら、要件が大きすぎます。

撮影待ち

`/task-breakdown` の結果
(3〜5個に割れた状態)

第6章 受講者がやること

ここから10分、この手順を2回まわします

割った結果は `session02-spec/tasks.md` に保存されています。ここでは2周だけ。残りは第7章で続けます。

1 tasks.md を開いて、まだ終わっていないタスクを1つ選ぶ

3 そのタスクを貼って送る

5 完了条件を満たしているか確かめる

7 tasks.md のそのタスクに印を付ける

2 新しいチャットを開く

4 返ってきたら diff を読む

6 両方 OK なら Keep

8 1 に戻る

2 の「新しいチャット」を飛ばさないでください。前のタスクの文脈が残っていると、頼んでいないところまで直され始めます。1タスク=1チャットです。

第6章 受講者がやること

毎回送るのは、この形

上の 3 で毎回送るものです。 `tasks.md` からコピーします。

(`tasks.md` からタスクの「やること」をコピー)

完了条件: (`tasks.md` からそのタスクの完了条件をコピー)

他の機能は変えないで。

返ってきたら `diff` を読む → 完了条件を満たしているか確かめる → `Keep`。それから次のタスクへ。

撮影待ち

タスク1つを渡した依頼文
(完了条件つき)

第6章 受講者がやること

diff を読んでから Keep する

撮影待ち

diff を読んで Keep するところ

第1回でやったことと同じです。ただし今日は、**diff を読んだあとに完了条件と照らします**。両方 OK なら Keep。

CURSOR 実践コース 第 2 回 / 全 4 回

CHAPTER 07 ・ 1:05 15分

開発ルールを決めて、効かせる

第6章で毎回書いた2行を、置いておきます。そして残りのタスクを、書かずに作ります。

第7章 解説

今日さわる3つ目の仕組み

仕組み	いつ効くか	置き場所	今日でいうと
Rules	いつも	.cursor/rules/*.mdc	これからやる
Skills	呼んだときだけ	.cursor/skills/...	requirements / task-breakdown
要件	その案件だけ	ふつうのファイル	ポーカーの要件

Skills と要件は今日さわりました。Rules だけがまだです。

第7章 解説

毎回書いたものを、置いておける

第6章で、タスクを渡すたびに毎回この2行を書きました。タスクの数だけ、同じことを書いています。

完了条件: (tasks.md からそのタスクの完了条件をコピー)
他の機能は変えないで。

毎回のプロンプトに書かなくて済ませる場所が `.cursor/rules/` です。

第 7 章 解説

ルールの書き方

`.cursor/rules/` に `.mdc` を置きます。Markdown の頭に設定が付いただけのものです。

```
---
description: ルール一覧に出る短い説明
globs: このパターンのファイルを触るときだけ効かせる
alwaysApply: true なら毎回必ず載せる
---

# 見出し

- やること
- やらないこと
```

書き方のコツ	中身
短く具体的に	守れる 5～15 行
やる / やらないを書く	曖昧な方針は守られない
alwaysApply を乱用しない	毎掲載るぶん文脈を食う

第7章 受講者がやること

ルールを1本書く (3分)

Agent に、次をそのまま送ってください。

`.cursor/rules/task-cycle.mdc` を作って。

`alwaysApply: true` にして。

内容は次の4行だけ。他は足さないで。

- 一度に着手するタスクは1つだけ
- 実装したら、そのタスクの完了条件と照らして確認する
- 完了条件を満たしたらそこで止まる。次のタスクへ勝手に進まない
- 依頼されていない機能を足さない

4行目に注目してください。「依頼されていない機能を足さない」 — 今日ずっと困っていたことです。

第7章 受講者がやること

2行を書かずに、続きを作る（9分）

第6章の続きです。残りのタスクを実装します。ただし今回は、これだけ送ります。

(tasks.md からタスクの「やること」だけをコピー)

	第6章	いま
毎回書くもの	やること + 完了条件 + 他は変えないで	やることだけ
完了条件は	プロンプトに書いた	tasks.md にある (AI が読む)
「他は変えないで」は	プロンプトに書いた	ルールにある

手順は第6章と同じです。tasks.md から1つ選ぶ → 新しいチャット → 送る → diff を読む → 完了条件と照らす → Keep → 印を付ける。

第7章

完了ライン

- ☐ 5枚のカードが配られて、画面に見える
- ☐ カードを選んで交換できる
- ☐ 役名が表示される
- ☐ 第5章で来た追加要求（役をつくっているカードに枠が付く）が入っている

3つ目まで行けば十分です。4つ目は挑戦枠（今日のゴールの 04）。

途中で止まっても、続きを誰かに頼めます。要件とタスクとルールが、全部文字になっているからです。神経衰弱の方は、続きをあなたにしか頼めません。そこが違います。

第7章 見本 ★

枠を見れば、合っているか分かります

見本 ・ 同じものにはなりません

撮影待ち

できたポーカー

(役をつくっているカードに枠が付いた状態)

第4章では、合っているかを**自分の記憶と照らす**しかありませんでした。いまは**画面を見れば分かります**。判定する基準を、頭の中から画面に出したんです。

第7章 コラム

早く終わった人へ (Design Mode)

見た目を直したくなったら、言葉で説明せずに「指す」方法があります。内蔵ブラウザで自分のゲームを開いた状態で押します。

操作	キー
Design Mode の切り替え	Ctrl+Shift+D (Mac は Cmd+Shift+D)
範囲を選ぶ	Shift + ドラッグ
選んだ要素をチャットに追加	Ctrl+L (Mac は Cmd+L)

向くのは見た目・余白・配置。役の判定などの計算ロジックには向きません（普通に Agent へ依頼する）。

CURSOR 実践コース 第 2 回 / 全 4 回

CHAPTER 08 ・ 1:20 10分

持ち帰るもの

使い分けの判断、そして次回。ここは短縮しません。

第8章

提出（30秒）

自分のポーカーの画面の
スクリーンショットを1枚だけ
チャットか共有フォルダに貼ってください。

カードに枠が付いていれば十分です。
勝敗まで動いていなくて大丈夫。

撮影待ち

提出物の見本

第8章

今日おぼえて帰ること

01

バンプコーディングは速い。ただしコントロールできない

狙えない／把握できない／確かめられない

02

要件は「他人に渡せる形」にすること

やること + やらないこと

03

判定できる形にしておくこと

頭の中の基準を、画面か完了条件に出す

第8章

使い分けの目安

こういうときは	やり方
小さい・使い捨て・自分だけが見る	バンプコーディング （神経衰弱くらいの規模）
決めることが多い・他人と作る・後から直す	仕様駆動開発 （ポーカーくらいの規模から）

境目は「規模」ではなく「決めることの量」です。決めることが増えるほど、コントロールを手放す量も増えます。

次回予告 ・ 第 3 回

チームで作りはじめる

自分たちでテーマを決めてアプリを作り始めます。
今日やった「要件を先に書く」を、チームでやるとどうなるか。
ブランチ・PR・レビューの最小フローも入れます。

第8章 次回に向けて

この2つ、いま確認します

1つ目は Cursor の画面だけで確認できます。今日ファイルを触っているので、ソース管理のアイコンに数字が付いていれば OK です。

- ☐ サイドバーの「ソース管理」に、今日の変更が出ている
- ☐ GitHub アカウントを持っている

どちらか欠けている人は**挙手**してください。第3回の進め方を決める材料にします。